

CS-306-MJ-P Core Java and Web Technology-I

T.Y.B.Sc.(C.S.) SEM-V | Q2 (Web Technology) - All 30 Slips with Solutions

Slip	Topic	Slip	Topic
1	College/School Registration Form	17	JS modules
2	Student Registration Form	18	Node HTTP server
3	Flexbox college homepage	19	async/await login
4	Change heading on button click	20	Promise divide
5	Image gallery (hover, grid)	21	Node path module
6	:hover button and link	22	fs async operations
7	:first-child / :last-child list	23	fs create/write/read/append
8	DOM image swap	24	Sync vs async fs
9	Arrow functions (calculator)	25	Directory mgmt + JSON
10	Template literals report	26	fs copy/rename/delete
11	Arrow fn total & average	27	Read-modify-write JSON
12	Arrow fn bill	28	List & filter files
13	Swap via array destructuring	29	fs.stat()
14	Object destructuring	30	Menu-driven file manager
15	Spread operator		
16	Rest parameter		

Q2) Design a College/School Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.

Reference screenshot:

My-Future College / School
Please Fill Below Detail In Student Registration Form

Student name : Min 3 & Max 10 letter is allow

Date of birth : dd-mm-yyyy Valid Date is allow

Mobile no. : Only integer is allow

Email id : Valid Email Id is allow

Password : Min 8 digit is allow

Gender : ☐ Male ☐ Female

Hobbies : ☐ Drawing ☐ Singing ☐ Dancing ☐ Sketch ☐ Sports

Course : Select Admission Course's

Student photo : No file chosen

Register Time: --:--

Address :

☐ I hereby declare that above information provided is correct

Answer - form1.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My-Future College / School</title>
<style>
body{font-family:"Times New Roman", serif;background:#fff;margin:0;padding:20px}
.box{width:640px;margin:auto;background:#ffffcc;padding:15px 25px}
h1{text-align:center;margin:5px 0 0}
.tag{text-align:center;font-size:12px;margin-bottom:15px}
table{width:100%;border-collapse:collapse}
td{padding:7px 4px;font-size:14px;vertical-align:middle}
td:first-child{width:28%;color:#333}
input[type=text],input[type=date],input[type=tel],input[type=email],
input[type=password],input[type=time],select{width:65%}
textarea{width:65%;height:70px}
.hint{color:red;font-size:11px}
.btnns{text-align:center;padding-top:10px}
button{padding:5px 30px;margin:0 8px;cursor:pointer}
input:invalid:not(:placeholder-shown){border:1px solid red}
input:valid:not(:placeholder-shown){border:1px solid green}
</style>
</head>
<body>
<div class="box">
<h1>My-Future College / School</h1>
<div class="tag">Please Fill Below Detail In Student Registration Form</div>
<form action="#" method="post">
<table>
<tr><td>Student name :</td>
<td><input type="text" name="sname" placeholder="Name" required
minlength="3" maxlength="10" pattern="[A-Za-z ]{3,10}">
<span class="hint">Min 3 & Max 10 letter is allow</span></td></tr>
<tr><td>Date of birth :</td>
<td><input type="date" name="dob" required max="2015-12-31">
<span class="hint">Valid Date is allow</span></td></tr>
<tr><td>Mobile no. :</td>
<td><input type="tel" name="mobile" placeholder="10 digit no." required
pattern="[0-9]{10}" maxlength="10" title="Only integers, 10 digits">
<span class="hint">Only integer is allow</span></td></tr>
<tr><td>Email id :</td>
<td><input type="email" name="email" placeholder="abc@mail.com" required>
```

```

        <span class="hint">Valid Email Id is allow</span></td></tr>
<tr><td>Password :</td>
    <td><input type="password" name="pwd" placeholder="Password" required minlength="8">
        <span class="hint">Min 8 digit is allow</span></td></tr>
<tr><td>Gender :</td>
    <td><input type="radio" name="gender" id="m" value="Male" required><label for="m">Male</label>
        <input type="radio" name="gender" id="f" value="Female"><label for="f">Female</label></td></tr>
<tr><td>Hobbies :</td>
    <td><input type="checkbox" name="hobby" value="Drawing">Drawing
        <input type="checkbox" name="hobby" value="Singing">Singing
        <input type="checkbox" name="hobby" value="Dancing">Dancing
        <input type="checkbox" name="hobby" value="Sketch">Sketch
        <input type="checkbox" name="hobby" value="Sports">Sports</td></tr>
<tr><td>Course :</td>
    <td><select name="course" required>
        <option value="">---- Select Admission Course's ----</option>
        <option>B.Sc. Computer Science</option>
        <option>BCA</option>
        <option>B.Com</option>
        <option>BBA</option></select></td></tr>
<tr><td>Student photo :</td>
    <td><input type="file" name="photo" accept="image/*" required></td></tr>
<tr><td>Register Time:</td>
    <td><input type="time" name="rtime" required></td></tr>
<tr><td>Address :</td>
    <td><textarea name="address" required minlength="10" maxlength="200"></textarea></td></tr>
<tr><td colspan="2"><input type="checkbox" id="dec" required>
    <label for="dec">I hereby declare that above information provided is correct</label></td></tr>
</table>
<div class="btns"><button type="submit">Register</button>
<button type="reset">Reset</button></div>
</form>
</div>
</body>
</html>

```

Save as form1.html and open in a browser. Expected output is the form shown above.

Q2) Design a Student Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.

Reference screenshot:

Answer - form2.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Student Registration Form</title>
<style>
  body{font-family:"Times New Roman",serif;margin:0;padding:20px}
  .box{width:600px;margin:auto;background:#ffe0dc;padding:10px 30px 20px}
  h2{text-align:center}
  table{width:100%}
  td{padding:7px 3px;font-size:13px}
  td:first-child{width:28%}
  .full{width:95%}
  .s{width:60px}
  select{width:70%}
  textarea{width:65%;height:70px}
  .c{text-align:center}
</style>
</head>
<body>
<div class="box">
<h2>Student Registration Form</h2>
<form action="#" method="post">
<table>
<tr><td>Roll no. :</td>
  <td><input class="full" type="number" name="roll" min="1" required></td></tr>
<tr><td>Student name :</td>
  <td><input type="text" name="fname" placeholder="First Name" required pattern="[A-Za-z]+">
    - <input type="text" name="lname" placeholder="Last Name" required pattern="[A-Za-z]+"></td></tr>
<tr><td>Father's name :</td>
  <td><input class="full" type="text" name="father" required pattern="[A-Za-z ]+"></td></tr>
<tr><td>Date of birth :</td>
  <td><input class="s" type="number" name="d" placeholder="Day" min="1" max="31" required> -
    <input class="s" type="number" name="m" placeholder="Month" min="1" max="12" required> -
    <input class="s" type="number" name="y" placeholder="Year" min="1990" max="2015" required>
    <i>(DD-MM-YYYY)</i></td></tr>
<tr><td>Mobile no. :</td>
  <td><input class="s" type="text" value="+91" readonly> -
    <input type="tel" name="mobile" pattern="[6-9][0-9]{9}" maxlength="10" required
      title="10 digit mobile number"></td></tr>
<tr><td>Email id :</td>
```

```

        <td><input class="full" type="email" name="email" required></td></tr>
<tr><td>Password :</td>
        <td><input class="full" type="password" name="pwd" minlength="8" required></td></tr>
<tr><td>Gender :</td>
        <td><input type="radio" name="g" value="Male" required>Male
            <input type="radio" name="g" value="Female">Female</td></tr>
<tr><td>Department :</td>
        <td><input type="checkbox" name="dept" value="CSE">CSE
            <input type="checkbox" name="dept" value="IT">IT
            <input type="checkbox" name="dept" value="ECE">ECE
            <input type="checkbox" name="dept" value="Civil">Civil
            <input type="checkbox" name="dept" value="Mech">Mech</td></tr>
<tr><td>Course :</td>
        <td><select name="course" required>
            <option value="">----- Select Current Course's -----</option>
            <option>FY</option><option>SY</option><option>TY</option></select></td></tr>
<tr><td>Student photo :</td>
        <td><input type="file" name="photo" accept="image/*" required></td></tr>
<tr><td>City :</td>
        <td><input class="full" type="text" name="city" required></td></tr>
<tr><td>Address :</td>
        <td><textarea name="address" required></textarea></td></tr>
<tr><td colspan="2" class="c"><button type="submit">Register</button></td></tr>
</table>
</form>
</div>
</body>
</html>

```

Save as form2.html and open in a browser. Expected output is the form shown above.

Q2) Create a responsive webpage using CSS Flexbox for a college homepage containing header, navigation, content and footer. Use appropriate Flexbox alignment and distribution properties.

Answer - college.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>College Homepage</title>
<style>
  *{box-sizing:border-box;margin:0;padding:0;font-family:Arial,sans-serif}
  body{display:flex;flex-direction:column;min-height:100vh}
  header{background:#1e3a8a;color:#fff;padding:20px;
    display:flex;justify-content:space-between;align-items:center}
  nav{background:#2563eb}
  nav ul{list-style:none;display:flex;justify-content:space-around;flex-wrap:wrap}
  nav a{display:block;padding:14px 20px;color:#fff;text-decoration:none}
  nav a:hover{background:#1d4ed8}
  main{flex:1;display:flex;flex-wrap:wrap;gap:20px;padding:20px;justify-content:center}
  .card{flex:1 1 250px;background:#f1f5f9;padding:20px;border-radius:8px}
  footer{background:#0f172a;color:#fff;padding:15px;
    display:flex;justify-content:center;align-items:center}
  @media(max-width:600px){
    header{flex-direction:column;gap:8px}
    nav ul{flex-direction:column}
  }
</style>
</head>
<body>
<header><h1>ABC College</h1><p>Excellence in Education</p></header>
<nav><ul>
  <li><a href="#">Home</a></li><li><a href="#">About</a></li>
  <li><a href="#">Courses</a></li><li><a href="#">Admissions</a></li>
  <li><a href="#">Contact</a></li>
</ul></nav>
<main>
  <div class="card"><h2>About Us</h2><p>We provide quality education since 1990.</p></div>
  <div class="card"><h2>Courses</h2><p>B.Sc. CS, BCA, B.Com, BBA.</p></div>
  <div class="card"><h2>Events</h2><p>Annual day, sports meet, tech fest.</p></div>
</main>
<footer><p>&copy; 2026 ABC College. All rights reserved.</p></footer>
</body>
</html>
```

SLIP 4 - Q2 [15 marks]

Q2) Create a webpage containing a heading and a button. When the button is clicked, change the heading text from "Hello! Welcome" to "Text Changed Successfully!".

Answer - change_text.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Change Heading</title>
</head>
<body style="text-align:center;margin-top:80px;font-family:Arial">
  <h1 id="heading">Hello! Welcome</h1>
  <button onclick="changeText()">Click Me</button>
  <script>
    function changeText() {
      document.getElementById("heading").innerText = "Text Changed Successfully!";
    }
  </script>
</body>
</html>
```

SLIP 5 - Q2 [15 marks]

Q2) Create an image gallery using HTML5 and CSS3 with hover effects, responsive images and Flexbox/Grid layout.

Answer - gallery.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Image Gallery</title>
<style>
  body{font-family:Arial,sans-serif;background:#111;color:#fff;margin:0;padding:20px}
  h1{text-align:center}
  .gallery{display:grid;gap:15px;
    grid-template-columns:repeat(auto-fit,minmax(220px,1fr))}
  .item{overflow:hidden;border-radius:10px;position:relative}
  .item img{width:100%;height:100%;display:block;object-fit:cover;
    transition:transform .4s,filter .4s}
  .item:hover img{transform:scale(1.15);filter:brightness(.7)}
  .item span{position:absolute;bottom:10px;left:10px;opacity:0;transition:.4s}
  .item:hover span{opacity:1}
</style>
</head>
<body>
<h1>My Gallery</h1>
<div class="gallery">
  <div class="item"><span>Nature</span></div>
  <div class="item"><span>Desk</span></div>
  <div class="item"><span>Mug</span></div>
  <div class="item"><span>Cat</span></div>
  <div class="item"><span>Sky</span></div>
  <div class="item"><span>Room</span></div>
</div>
</body>
</html>
```

Internet needed for the sample images. For offline use, replace the src values with local files (images/1.jpg ...).

SLIP 6 - Q2 [15 marks]

Q2) Create a webpage containing a button and link. Change their appearance when the mouse pointer moves over them. (Use :hover Pseudo-Class)

Answer - hover.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Hover Effect</title>
```

```

<style>
  body{text-align:center;margin-top:80px;font-family:Arial}
  button{padding:12px 30px;font-size:16px;background:#2563eb;color:#fff;
    border:none;border-radius:6px;cursor:pointer;transition:.3s}
  button:hover{background:#dc2626;transform:scale(1.1)}
  a{display:block;margin-top:40px;font-size:20px;color:#2563eb;text-decoration:none}
  a:hover{color:#fff;background:#16a34a;text-decoration:underline;
    padding:6px 12px;border-radius:4px}
</style>
</head>
<body>
  <button>Hover Me</button>
  <a href="https://www.google.com">Hover over this link</a>
</body>
</html>

```

SLIP 7 - Q2 [15 marks]

Q2) Create an unordered list of Programming Language names and apply different styles to the first and last list items. (Use :first-child and :last-child)

Answer - list.html

```

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Programming Languages</title>
<style>
  body{font-family:Arial;margin:40px}
  ul{list-style-type:square;font-size:18px}
  li{padding:4px}
  li:first-child{color:#fff;background:#2563eb;font-weight:bold;font-size:22px}
  li:last-child{color:#fff;background:#dc2626;font-style:italic;font-size:22px}
</style>
</head>
<body>
  <h2>Programming Languages</h2>
  <ul>
    <li>Java</li>
    <li>Python</li>
    <li>C++</li>
    <li>JavaScript</li>
    <li>PHP</li>
  </ul>
</body>
</html>

```

SLIP 8 - Q2 [15 marks]

Q2) Create a webpage using JavaScript DOM manipulation and event handling with an image and a button. When the button is clicked, replace the displayed image with another image.

Answer - image_swap.html

```

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Change Image</title>
</head>
<body style="text-align:center;font-family:Arial">
  <h2>Click the button to change image</h2>
  <br><br>
  <button id="btn">Change Image</button>
  <script>
    const pic = document.getElementById("pic");
    const btn = document.getElementById("btn");
    const img1 = "https://picsum.photos/id/10/400/300";
    const img2 = "https://picsum.photos/id/20/400/300";
    btn.addEventListener("click", () => {
      pic.src = pic.src === img1 ? img2 : img1;
    });
  </script>
</body>
</html>

```

Replace URLs with local files (a.jpg, b.jpg) if offline.

SLIP 9 - Q2 [15 marks]

Q2) Write a JavaScript program to demonstrate Arrow Functions for addition, subtraction, multiplication and division.

Answer - arrow.js

```
const add = (a, b) => a + b;
const sub = (a, b) => a - b;
const mul = (a, b) => a * b;
const div = (a, b) => (b === 0 ? "Cannot divide by zero" : a / b);

const x = 20, y = 5;
console.log("Addition      :", add(x, y));
console.log("Subtraction   :", sub(x, y));
console.log("Multiplication:", mul(x, y));
console.log("Division      :", div(x, y));
console.log("Division by 0  :", div(x, 0));

/* Output
Addition      : 25
Subtraction   : 15
Multiplication : 100
Division      : 4
Division by 0  : Cannot divide by zero */
Run: node arrow.js
```

SLIP 10 - Q2 [15 marks]

Q2) Write a JavaScript program to store student information and marks in variables and generate a formatted student report using template literals.

Answer - report.js

```
const name = "Max";
const rollNo = 101;
const className = "TYBSc (CS)";
const m1 = 78, m2 = 85, m3 = 90;

const total = m1 + m2 + m3;
const percentage = (total / 3).toFixed(2);
const result = percentage >= 40 ? "Pass" : "Fail";

const report = `
===== STUDENT REPORT =====
Name      : ${name}
Roll No   : ${rollNo}
Class     : ${className}
-----
Subject 1 : ${m1}
Subject 2 : ${m2}
Subject 3 : ${m3}
-----
Total      : ${total}
Percentage: ${percentage}%
Result     : ${result}
=====`;
console.log(report);
Run: node report.js
```

SLIP 11 - Q2 [15 marks]

Q2) Write a JavaScript program using an arrow function to calculate the total and average marks of three subjects. Display the result using template literals.

Answer - marks.js

```
const calculate = (m1, m2, m3) => {
  const total = m1 + m2 + m3;
  const average = total / 3;
  return { total, average };
};

const [s1, s2, s3] = [70, 80, 90];
```

```
const { total, average } = calculate(s1, s2, s3);

console.log(`Marks    : ${s1}, ${s2}, ${s3}`);
console.log(`Total    : ${total}`);
console.log(`Average  : ${average.toFixed(2)}`);

Run: node marks.js
```

SLIP 12 - Q2 [15 marks]

Q2) Write a JavaScript program using an arrow function to calculate the total price of a product. Display the bill using template literals and string interpolation.

Answer - bill.js

```
const totalPrice = (price, qty, gstPercent) => {
  const amount = price * qty;
  const gst = (amount * gstPercent) / 100;
  return { amount, gst, total: amount + gst };
};

const product = "Keyboard";
const price = 1500, qty = 3, gstPercent = 18;
const { amount, gst, total } = totalPrice(price, qty, gstPercent);

console.log(`
***** BILL *****
Product   : ${product}
Price     : Rs. ${price}
Quantity  : ${qty}
Amount    : Rs. ${amount}
GST (${gstPercent}%) : Rs. ${gst}
-----
Total     : Rs. ${total}
*****`);
```

Run: node bill.js

SLIP 13 - Q2 [15 marks]

Q2) Write a JavaScript program to swap two numbers without using a third variable using array destructuring.

Answer - swap.js

```
let a = 10;
let b = 20;
console.log(`Before swap: a = ${a}, b = ${b}`);

[a, b] = [b, a]; // array destructuring swap

console.log(`After swap : a = ${a}, b = ${b}`);

/* Output
Before swap: a = 10, b = 20
After swap : a = 20, b = 10 */
```

Run: node swap.js

SLIP 14 - Q2 [15 marks]

Q2) Create an object containing employee details such as name, department, and salary. Use object destructuring to extract and display the values.

Answer - employee.js

```
const employee = {
  name: "Rahul Patil",
  department: "IT",
  salary: 45000
};

const { name, department, salary } = employee;

console.log("Name      :", name);
console.log("Department:", department);
console.log("Salary     :", salary);

// with renaming and default value
```

```
const { name: empName, city = "Pune" } = employee;
console.log(empName, "-", city);
```

Run: *node employee.js*

SLIP 15 - Q2 [15 marks]

Q2) Write a JavaScript program to create a copy of an existing array using the spread operator and add a new element to the copied array.

Answer - spread.js

```
const original = [1, 2, 3, 4, 5];
const copy = [...original]; // copy using spread
copy.push(6); // add new element to the copy

console.log("Original array :", original);
console.log("Copied array :", copy);

// adding at the beginning / middle using spread
const another = [0, ...original, 99];
console.log("With extras :", another);

/* Output
Original array : [ 1, 2, 3, 4, 5 ]
Copied array : [ 1, 2, 3, 4, 5, 6 ]
With extras : [ 0, 1, 2, 3, 4, 5, 99 ] */
```

Run: *node spread.js*

SLIP 16 - Q2 [15 marks]

Q2) Write a JavaScript program using a rest parameter to accept any number of values and calculate their sum.

Answer - rest.js

```
function sum(...numbers) {
  let total = 0;
  for (const n of numbers) total += n;
  return total;
}

console.log("Sum of 1,2,3 :", sum(1, 2, 3));
console.log("Sum of 10,20,30,40 :", sum(10, 20, 30, 40));
console.log("Sum of no values :", sum());

// arrow function version
const sum2 = (...nums) => nums.reduce((t, n) => t + n, 0);
console.log("Arrow version :", sum2(5, 5, 5));
```

Run: *node rest.js*

SLIP 17 - Q2 [15 marks]

Q2) Create a JavaScript module containing arithmetic operation functions. Export the functions and import them into another JavaScript file to perform all operations.

Answer - mathOps.mjs

```
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
export const multiply = (a, b) => a * b;
export const divide = (a, b) => {
  if (b === 0) throw new Error("Division by zero");
  return a / b;
};
```

Answer - main.mjs

```
import { add, subtract, multiply, divide } from "./mathOps.mjs";

const a = 20, b = 4;
console.log("Add :", add(a, b));
console.log("Subtract :", subtract(a, b));
console.log("Multiply :", multiply(a, b));
console.log("Divide :", divide(a, b));
```

Keep both files in one folder. Run: *node main.mjs* (.mjs enables ES modules; or use .js with "type":"module" in package.json).

SLIP 18 - Q2 [15 marks]

Q2) Create a Node.js HTTP server that sends an appropriate response to a client request.

Answer - server.js

```
const http = require("http");

const server = http.createServer((req, res) => {
  if (req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end("<h1>Welcome to Node.js Server</h1>");
  } else if (req.url === "/about") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("This is the About page");
  } else if (req.url === "/api") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ message: "Hello from API", status: "ok" }));
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("404 - Page Not Found");
  }
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000/");
});
```

Run: `node server.js`, then open `http://localhost:3000` in the browser.

SLIP 19 - Q2 [15 marks]

Q2) Write a JavaScript program using `async/await` to simulate user login. If the username and password are correct, display a success message; otherwise, handle the error using `try-catch`.

Answer - login.js

```
const login = (username, password) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (username === "admin" && password === "1234") {
        resolve("Login successful! Welcome, " + username);
      } else {
        reject(new Error("Invalid username or password"));
      }
    }, 1000);
  });
};

async function main(user, pass) {
  try {
    console.log("Logging in...");
    const msg = await login(user, pass);
    console.log(msg);
  } catch (err) {
    console.log("Error:", err.message);
  }
}

main("admin", "1234"); // success
setTimeout(() => main("admin", "wrong"), 1500); // failure
```

Run: `node login.js`

SLIP 20 - Q2 [15 marks]

Q2) Write a JavaScript program using a `Promise` to divide two numbers. If the denominator is zero, reject the `Promise` and handle the error.

Answer - divide.js

```
const divide = (a, b) => {
  return new Promise((resolve, reject) => {
    if (b === 0) {
      reject("Error: Denominator cannot be zero");
    } else {
      resolve(a / b);
    }
  });
}
```

```

});
};

divide(20, 4)
  .then(result => console.log("Result:", result))
  .catch(err => console.log(err));

divide(10, 0)
  .then(result => console.log("Result:", result))
  .catch(err => console.log(err))
  .finally(() => console.log("Done"));

/* Output
Result: 5
Error: Denominator cannot be zero
Done */

Run: node divide.js

```

SLIP 21 - Q2 [15 marks]

Q2) Write a Node.js program using the path module to perform operations such as joining, resolving and extracting file/directory information.

Answer - pathdemo.js

```

const path = require("path");

const filePath = "/home/user/projects/app/index.html";

console.log("join      :", path.join("/home", "user", "docs", "file.txt"));
console.log("resolve   :", path.resolve("docs", "file.txt"));
console.log("basename  :", path.basename(filePath));
console.log("dirname   :", path.dirname(filePath));
console.log("extname    :", path.extname(filePath));
console.log("isAbsolute:", path.isAbsolute(filePath));
console.log("normalize:", path.normalize("/home//user/./docs/./a.txt"));
console.log("parse     :", path.parse(filePath));
console.log("format     :", path.format({ dir: "/home/user", base: "test.js" }));
console.log("relative  :", path.relative("/home/user/a", "/home/user/b/c"));
console.log("sep       :", path.sep);

Run: node pathdemo.js

```

SLIP 22 - Q2 [15 marks]

Q2) Write a Node.js program using the fs module to perform file operations asynchronously.

Answer - fs_async.js

```

const fs = require("fs");

fs.writeFile("demo.txt", "Hello Node.js\n", (err) => {
  if (err) return console.log("Write error:", err.message);
  console.log("File created and written");

  fs.appendFile("demo.txt", "Appended line\n", (err) => {
    if (err) return console.log("Append error:", err.message);
    console.log("Data appended");

    fs.readFile("demo.txt", "utf8", (err, data) => {
      if (err) return console.log("Read error:", err.message);
      console.log("File content:\n" + data);

      fs.unlink("demo.txt", (err) => {
        if (err) return console.log("Delete error:", err.message);
        console.log("File deleted");
      });
    });
  });
});

console.log("This line prints first (non-blocking)");

Run: node fs_async.js

```

SLIP 23 - Q2 [15 marks]

Q2) Write a Node.js program to create, write, read and append data to a file using the fs module.

Answer - fileops.js

```
const fs = require("fs");
const file = "student.txt";

// create + write (overwrites if exists)
fs.writeFileSync(file, "Name: Max\nClass: TYBSc CS\n");
console.log("1. File created and data written");

// read
console.log("2. Content:\n" + fs.readFileSync(file, "utf8"));

// append
fs.appendFileSync(file, "City: Pune\n");
console.log("3. Data appended");

// read again
console.log("4. Updated content:\n" + fs.readFileSync(file, "utf8"));
Run: node fileops.js
```

SLIP 24 - Q2 [15 marks]

Q2) Write a Node.js program to demonstrate synchronous and asynchronous file operations and compare their execution.

Answer - compare.js

```
const fs = require("fs");
fs.writeFileSync("sample.txt", "Sample data for comparison\n");

console.log("---- Synchronous ----");
console.log("Start");
const data = fs.readFileSync("sample.txt", "utf8"); // blocks here
console.log("Sync data:", data.trim());
console.log("End");

console.log("---- Asynchronous ----");
console.log("Start");
fs.readFile("sample.txt", "utf8", (err, d) => { // does not block
  if (err) return console.log(err.message);
  console.log("Async data:", d.trim());
  fs.unlinkSync("sample.txt");
});
console.log("End (printed BEFORE async data)");

/* Sync : Start -> data -> End (in order, program waits)
   Async: Start -> End -> data (callback runs later) */
Run: node compare.js
```

SLIP 25 - Q2 [15 marks]

Q2) Write a Node.js program to perform directory management operations such as creating, listing, renaming and deleting directories. Also demonstrate JSON parsing and writing structured JSON data.

Answer - dirjson.js

```
const fs = require("fs");
const path = require("path");

// ---- Directory operations ----
fs.mkdirSync("mydir", { recursive: true });
console.log("Directory created");

fs.writeFileSync(path.join("mydir", "a.txt"), "hello");
console.log("Listing:", fs.readdirSync("mydir"));

fs.renameSync("mydir", "newdir");
console.log("Directory renamed to newdir");

// ---- JSON operations ----
const student = { id: 1, name: "Max", subjects: ["Java", "Web"], passed: true };
const jsonPath = path.join("newdir", "student.json");
```

```

fs.writeFileSync(jsonPath, JSON.stringify(student, null, 2)); // write
console.log("JSON written");

const obj = JSON.parse(fs.readFileSync(jsonPath, "utf8")); // parse
console.log("Parsed name:", obj.name, "| subjects:", obj.subjects.join(", "));

// ---- Delete directory ----
fs.rmSync("newdir", { recursive: true, force: true });
console.log("Directory deleted");
Run: node dirjson.js

```

SLIP 26 - Q2 [15 marks]

Q2) Write a Node.js program using the fs module to copy, rename and delete files asynchronously. Display appropriate success and error messages.

Answer - filemgr.js

```

const fs = require("fs");

fs.writeFileSync("original.txt", "Some content");

// copy
fs.copyFile("original.txt", "copy.txt", (err) => {
  if (err) return console.log("Copy failed:", err.message);
  console.log("SUCCESS: File copied");

  // rename
  fs.rename("copy.txt", "renamed.txt", (err) => {
    if (err) return console.log("Rename failed:", err.message);
    console.log("SUCCESS: File renamed");

    // delete
    fs.unlink("renamed.txt", (err) => {
      if (err) return console.log("Delete failed:", err.message);
      console.log("SUCCESS: File deleted");

      // error demo - file no longer exists
      fs.unlink("renamed.txt", (err) => {
        if (err) console.log("ERROR:", err.message);
      });
    });
  });
});
Run: node filemgr.js

```

SLIP 27 - Q2 [15 marks]

Q2) Write a Node.js program to read a JSON file, modify selected properties of the JSON object, and write the updated JSON data back to the file using the fs module.

Answer - modjson.js

```

const fs = require("fs");
const file = "data.json";

// create sample file if not present
if (!fs.existsSync(file)) {
  fs.writeFileSync(file, JSON.stringify(
    { name: "Max", age: 20, city: "Pune", marks: 70 }, null, 2));
}

fs.readFile(file, "utf8", (err, data) => {
  if (err) return console.log("Read error:", err.message);

  const obj = JSON.parse(data);
  console.log("Before:", obj);

  obj.age = 21; // modify
  obj.marks = 85; // modify
  obj.course = "TYBSc CS"; // add new property

  fs.writeFile(file, JSON.stringify(obj, null, 2), (err) => {
    if (err) return console.log("Write error:", err.message);
    console.log("After :", obj);
    console.log("JSON file updated successfully");
  });
});

```

```
});
});
Run: node modjson.js
```

SLIP 28 - Q2 [15 marks]

Q2) Create a Node.js program to list files in a specified directory, filter files based on their extensions (such as .txt, .json and .js), and display the matching file names.

Answer - listfiles.js

```
const fs = require("fs");
const path = require("path");

const dir = process.argv[2] || "."; // node listfiles.js <dir>
const allowed = [".txt", ".json", ".js"];

fs.readdir(dir, (err, files) => {
  if (err) return console.log("Error:", err.message);

  const matched = files.filter(f => allowed.includes(path.extname(f).toLowerCase()));

  console.log(`Directory: ${path.resolve(dir)}`);
  if (matched.length === 0) return console.log("No matching files found");

  allowed.forEach(ext => {
    const list = matched.filter(f => path.extname(f).toLowerCase() === ext);
    console.log(`\n${ext} files (${list.length}):`);
    list.forEach(f => console.log("  " + f));
  });
});
```

Run: node listfiles.js (or node listfiles.js path/to/folder)

SLIP 29 - Q2 [15 marks]

Q2) Write a Node.js program to demonstrate file statistics using fs.stat(), including file size, file type, creation/access/modification information, and whether the path represents a file or directory.

Answer - stats.js

```
const fs = require("fs");

function showStats(p) {
  fs.stat(p, (err, s) => {
    if (err) return console.log("Error:", err.message);
    console.log("\n=== " + p + " ===");
    console.log("Size           :", s.size, "bytes");
    console.log("Is File           :", s.isFile());
    console.log("Is Directory      :", s.isDirectory());
    console.log("Type             :", s.isFile() ? "File" : s.isDirectory() ? "Directory" : "Other");
    console.log("Created (birth)  :", s.birthtime);
    console.log("Last Accessed    :", s.atime);
    console.log("Last Modified    :", s.mtime);
    console.log("Status Changed   :", s.ctime);
    console.log("Permissions      :", (s.mode & 0o777).toString(8));
  });
}
```

```
fs.writeFileSync("test.txt", "Hello stats");
showStats("test.txt"); // file
showStats(".");        // directory
showStats("nofile.txt"); // error case
```

Run: node stats.js

SLIP 30 - Q2 [15 marks]

Q2) Create a Node.js mini file-management application using the fs and path modules that allows the user to create, read, update, rename and delete text files through a menu-driven interface.

Answer - filemanager.js

```
const fs = require("fs");
const path = require("path");
const readline = require("readline");
const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
```



```

const ask = (q) => new Promise((res) => rl.question(q, res));
const base = path.join(__dirname, "files");
if (!fs.existsSync(base)) fs.mkdirSync(base);
const fp = (n) => path.join(base, path.basename(n));

async function main() {
  while (true) {
    console.log("\n===== FILE MANAGER =====");
    console.log("1. Create 2. Read 3. Update 4. Rename 5. Delete 6. List 7. Exit");
    const ch = await ask("Enter choice: ");
    try {
      if (ch === "1") {
        const n = await ask("File name: ");
        const c = await ask("Content: ");
        fs.writeFileSync(fp(n), c);
        console.log("File created");
      } else if (ch === "2") {
        const n = await ask("File name: ");
        console.log("---\n" + fs.readFileSync(fp(n), "utf8") + "\n---");
      } else if (ch === "3") {
        const n = await ask("File name: ");
        const c = await ask("Text to append: ");
        fs.appendFileSync(fp(n), "\n" + c);
        console.log("File updated");
      } else if (ch === "4") {
        const o = await ask("Old name: ");
        const nn = await ask("New name: ");
        fs.renameSync(fp(o), fp(nn));
        console.log("File renamed");
      } else if (ch === "5") {
        const n = await ask("File name: ");
        fs.unlinkSync(fp(n));
        console.log("File deleted");
      } else if (ch === "6") {
        console.log(fs.readdirSync(base));
      } else if (ch === "7") {
        console.log("Goodbye!");
        rl.close();
        return;
      } else {
        console.log("Invalid choice");
      }
    } catch (e) {
      console.log("Error:", e.message);
    }
  }
}
main();

```

Run: `node filemanager.js` (files are stored in a 'files' folder next to the script)